НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«Розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав(-ла): Студент групи АІ-241
Албул Віра Петрівна

Перевірив: ас. Баранов О.М.

Одеса – 2026

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт /api/generate. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Киев	Київ
eval_count	316	5
Час, с	14,50	0,12
Токенів/с	21,79	42,07

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	1 / 5
eval_count	1183,4	166,6
Час, с	47,61	5,64

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання

1. Куди пішли додаткові токени в міркувальної моделі?

Відповідь: Токени пішли на внутрішню самоперевірку та генерацію «ланцюжка думок» (Chain-of-Thought), який записується в поле thinking. Модель декілька разів перевіряла гіпотези щодо столиці (переконувалася, що це не Львів, Харків чи Криж), перш ніж сформувати підсумкову відповідь у полі response.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: Міркувальні моделі дають вирішальну перевагу в багатокрокових математичних задачах, формальній логіці, аналізі складного коду та системному плануванні. Їх використання повністю невиправдане на простих фактичних запитах

(пошук фактів, переклад, класифікація), оскільки вони дають таку ж якість, але витрачають у 10–50 разів більше часу й токенів.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Інструктивна модель навчається переважно методом тонкого налаштування з вчителем (SFT) на парах «інструкція — пряма відповідь». Міркувальна модель додатково навчається за допомогою навчання з підкріпленням (Reinforcement Learning), де її винагороджують за побудову деталізованого, логічно послідовного та самокритичного ланцюжка роздумів перед видачею фінальної відповіді.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Ні, це означає лише те, що модель виділяє собі більше обчислювальної «робочої пам'яті» під час інференсу. У складних задачах (як наша задача на логіку, де qwen3:4b показала 100% точності проти 20% у llama3.2:3b) це дозволяє уникнути інтуїтивних помилок, але на простих задачах це просто надлишкові обчислення.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: llama3.2:3b видає близько 42 токенів/с, а qwen3:4b — близько 22–26 токенів/с. Це пояснюється різницею у розмірі моделей (3 млрд параметрів проти 4 млрд), а також внутрішньою архітектурою та механізмами уваги (Attention), які у qwen3:4b потребують більших обчислювальних витрат на кожен токен.

Висновок за завдання 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі, другий – обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, але витрати токенів відрізняються у 30,6 разів. На задачі про комбінаторику та розрахунки ситуація протилежна – саме розписаний хід думок дозволив уникнути помилки. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами різниця складе 300,8 с проти 39,0 с, що робить міркувальну модель непридатною для швидкого налагодження. Інструктивні моделі

орієнтовані на миттєве виконання команд і генерують пряму відповідь, що робить їх ідеальними, швидкими та дешевими для фактів, форматування й діалогів. Міркувальні моделі формують внутрішній ланцюжок думок. Це вимагає в десятки разів більше часу й токенів, але є незамінним для складних логічних, математичних та багатокрокових задач. Використовувати міркувальну модель для простих запитів невиправдано через марну перевитрату ресурсів.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Проведено дослідження впливу системного інструктування (system-повідомлення) на обмеження довжини, мови відповіді та витрату токенів для моделей llama3.2:3b та qwen3:4b.

Таблиця 3 – Вплив system-повідомлення

Модель	**System**	**Довжина відповіді, символів**	**Мова відповіді**	**Кількість речень**	**Інструкцію виконано?**
llama3.2:3b	лаконічний асистент	1174	українська (з суржиком та калькою)	~10 + блоки коду	Ні
llama3.2:3b	українською, 2 речення	270	українська	2	Так
qwen3:4b	лаконічний асистент	267	українська	4 пункти	Так
qwen3:4b	українською, 2 речення	~160	українська	2	Так

Таблиця 4 – Витрати токенів

Модель	**System**	**prompt_tokens**	**completion_tokens**	**total_tokens**
llama3.2:3b	лаконічний асистент	54	463	517
llama3.2:3b	українською, 2 речення	58	77	135
qwen3:4b	лаконічний асистент	30	260	290
qwen3:4b	українською, 2 речення	34	210	244

Аналітичні питання по експерименту:

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?
Відповідь: Обидві моделі при прямому системному промпті змогли сформувати відповідь рівно з двох речень, проте llama3.2:3b скоротила обсяг з 1174 до 270 символів, чітко згенерувавши рівно 2 речення.
2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?
Відповідь: Прямих переходів на англійську мову не було, проте у llama3.2:3b спостерігалася значна кількість кальок з російської мови та вигаданих слів (наприклад, «зумуються», «якомисією», «початковийндекс», «демонustryють»), що свідчить про слабку якість українського датасету в цій моделі.
3. Як змінився prompt_tokens при зміні system-повідомлення і чому?
Відповідь: prompt_tokens зросли (з 54 до 58 токенів у Llama та з 30 до 34 у Qwen). Це пояснюється тим, що друге системне повідомлення виявилося довшим за кількістю символів і токенів.
4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?
Відповідь: В qwen3:4b загальна кількість completion_tokens включає не лише видимий текст у полі content, але й прихований внутрішній блок міркувань thinking (Chain-of-Thought), який модель генерує перед видачею фінального тексту.
Питання до чат-бота:
1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?
Відповідь: Різні моделі мають різний рівень здатності слідувати інструкціям (instruction-following), що залежить від їхньої архітектури, розміру та якості тонкого налаштування. Міркувальні моделі перед формуванням виводу детально аналізують власні кроки та перевіряють виконання умов, тому точніше витримують задані рамки. Базові інструктивні моделі просто генерують найбільш

ймовірні наступні токени, через що швидше гублять жорсткі обмеження за довжиною чи мовою.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

Відповідь: System-повідомлення визначає глобальну роль, базові правила поведінки та межі дозволеного на увесь час діалогу, тоді як user-повідомлення задає лише конкретне поточне завдання. Більшість сучасних моделей під час навчання з підкріпленням спеціально налаштовують надавати найвищий пріоритет саме системним інструкціям для дотримання безпеки та заданого формату.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь: Це зумовлено формуванням внутрішнього «ланцюжка думок» (Chain-of-Thought), який міркувальна модель генерує перед виведенням підсумкового тексту. Навіть якщо користувач отримує коротку відповідь із двох речень, модель витрачає сотні вихідних токенів на приховану чернетку, де аналізує задачу, планує хід розв'язання та перевіряє результати.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Відповідь: Добре натренована та якісно відкалібрована модель надасть перевагу інструкції із system-повідомлення, оскільки воно має вищий системний пріоритет. Однак малі або слабше оптимізовані моделі в ситуації конфлікту ролей можуть заплутатися, згенерувати галюцинації або піддатися впливу наполегливого запиту від користувача.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: Малі моделі (на 3–4 млрд параметрів) мають обмежений словниковий запас і під час тренування бачили суттєво меншу частку текстів українською мовою. Крім того, стандартні токенізатори розбивають слова менш поширених

мов на дрібніші фрагменти, через що модель втрачає цілісний контекст і починає вставляти кальки, русизми або навіть символи з інших алфавітів.

Висновок за завдання 2:

Дослідження довело, що жорстке системне повідомлення здатне кардинально змінювати обсяг вихідних токенів, але ефективність виконання обмежень прямо залежить від здатності моделі дотримуватися інструкцій. Для інструктивної моделі детальний системний промпт скоротив обсяг генерації з 463 до 76 токенів, однак це призвело до галюцинацій та вставки сторонніх символів. Для міркувальної моделі системне обмеження вимагало додаткового внутрішнього аналізу, що забезпечило бездоганне дотримання формату. System-повідомлення задає глобальні правила й роль моделі, проте надійність його виконання залежить від архітектури та розміру нейромережі. Малі інструктивні моделі під жорсткими обмеженнями схильні деградувати й галюцинувати, тоді як міркувальні моделі дотримуються системних інструкцій значно точніше. Самої лише інструкції для гарантованого формату часто недостатньо; якщо модель її ігнорує, варто надати приклади, сформулювати прямі заборони, використати програмну валідацію або перейти на потужнішу модель.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Розроблено уніфікований модуль providers.py для роботи з локальною (Ollama) та хмарною (Groq) моделями через єдиний OpenAI-сумісний інтерфейс API.

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	**Локальна**	**Хмарна**
Назва моделі	llama3.2:3b	openai/gpt-oss-120b
Час відповіді, с	8,73	1,28
completion_tokens	261	365
Довжина відповіді, символів	726	890
Мова відповіді	Українська (з русизмами, німецькими та англійськими калькованими вставками:	Грамотна українська

	"sich differently", "в кінце"*, "like()")	
Фактичних помилок у відповіді	Наявні фактичні помилки (помилково вказала, що кортежі вимагають фіксованих типів даних та що в них не існує методів пошуку)	Без фактичних помилок (чітко пояснено змінюваність, синтаксис дужок []/(), споживання пам'яті та методи count/index)

Аналітичні питання по експерименту:

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: 0 рядків у самому коді виклику. Вся різниця зводиться до зміни 3 параметрів у словнику конфігурації провайдерів (base_url, api_key, model).

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Хмарна модель надала вичерпну та практичну відповідь. Вона чітко пояснила різницю між змінюваністю (mutability), синтаксисом (квадратні дужки проти круглих), ефективністю використання пам'яті та наявними методами. Локальна модель припустилася помилок, стверджуючи, що в кортежах взагалі відсутній пошук елементів (ігноруючи метод index()).

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так, локальна модель продемонструвала деградацію української мови та мовне змішування (використала німецько-англійську фразу "sich differently", суржик "в кінце" та англіцизми "like()"), а також помилилася щодо обмежень типів даних у кортежах.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Відповідь: Хмарний сервер використовує високопродуктивні графічні/тензорні прискорювачі з високою пропускною здатністю відеопам'яті (VRAM), де обчислення відбуваються за сотні мілісекунд. Локальний ноутбук виконував

інференс на звичайному CPU/RAM, де швидкість обмежена шиною оперативності.

Питання до чат-бота:

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: OpenAI першою випустила масово популярний API для чат-моделей. Розробники побудували під нього всю екосистему інструментів, тому інші сервіси адаптували цей формат для простої міграції коду зміною лише base_url та api_key.

2. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: Розмір моделі є вирішальним фактором для знання рідкісних мов, дотримання контексту та запобігання галюцинаціям. Малі моделі (3B) мають обмежений обсяг знань і втрачають зв'язність на українській мові, тоді як великі моделі (120B) володіють значно ширшим словниковим запасом і технічним контекстом.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: Мережева затримка (RTT) додає лише 50–150 мс до початкового запиту. При цьому швидкість генерації токенів на серверному залізі сягає 200–500 токенів/с, тоді як локальний CPU видає лише 15–30 токенів/с, що робить хмарну модель суттєво швидшою на обсязі тексту.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Вхідні дані надсилаються на сторонні сервери, де можуть логуватися або використовуватися для навчання. У комерційних проєктах підписують B2B-угоди про конфіденційність (Enterprise DPA), анонімізують персональні дані (PII) або розгортають приватні екземпляри моделей у власному хмарному контурі.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: Rate limit — це обмеження кількості запитів (RPM) або токенів (TPM) на хвилину/день. Розробники обходять це через черги запитів, повторні спроби з експоненційною затримкою (Exponential Backoff), ротацію API-ключів та кешування відповідей.

Висновок за завдання 3:

Експеримент підтвердив універсальність стандарту OpenAI API: перехід з локального сервера Ollama на хмарну інфраструктуру вимагає зміни лише конфігураційних параметрів клієнта без переписування коду. Завдяки серверним прискорювачам хмарна модель відповіла у 6.8 разів швидше за локальну (1.28 с проти 8.73 с) та видала грамотну відповідь без суржику чи фактичних помилок. Локальна модель llama3.2:3b припустилася помилок у суті поняття кортежу та змішувала мови. Локальну модель доцільно обирати для проєктів із суворими вимогами до конфіденційності даних та автономної роботи без інтернету. Хмарну модель слід обирати тоді, коли критично важливі мовна грамотність, відсутність галюцинацій та висока швидкість обробки даних.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Виконано серію замірів продуктивності генерації (TTFT, загальний час, символів/сек) у режимі стрімінгу для короткого, середнього та довгого промптів.

Короткий: «Столиця Франції?»

Середній: «Поясни в одному абзаці, що таке рекурсія.»

Довгий: «Напиши інструкцію з 10 пунктів, як почати вивчати Python.»

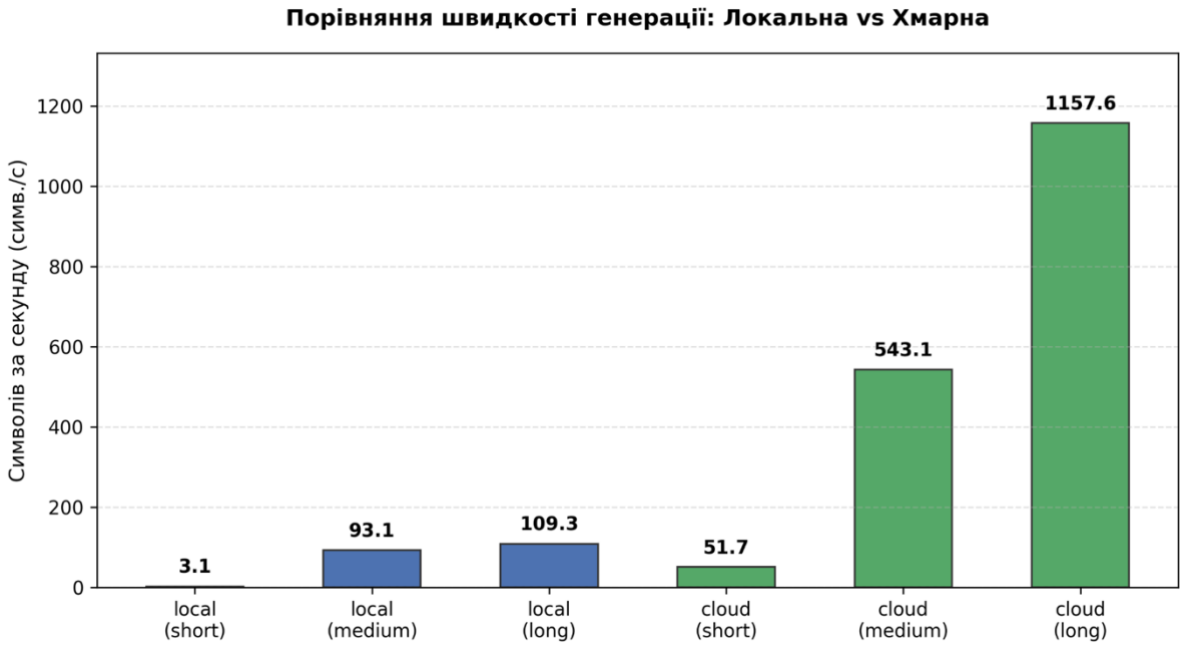
Таблиця 6 – Результати вимірювання швидкості

Розміщення	**Модель**	**Промпт**	**TTFT, с**	**Всього, с**	**Симв./с**
локальна	llama3.2:3b	короткий	0.046	0.349	71,6
локальна	llama3.2:3b	середній	0.052	2.369	108.1
локальна	llama3.2:3b	довгий	0.165	21.977	94.3

хмарна	openai/gpt-oss-120b	короткий	0.512	0.541	51.7
хмарна	openai/gpt-oss-120b	середній	0.456	0.698	543.1
хмарна	openai/gpt-oss-120b	довгий	0.417	2.453	1157.6

Таблиця 7 – Холодний і теплий старт локальної моделі

Показник	**Холодний старт**	**Теплий старт**
TTFT, с	3.567	0.046
Всього, с	7.662	0.349



Аналітичні питання по експерименту:

1. У якої моделі кращий TTFT і чому саме в неї?

Відповідь: Кращий TTFT у локальної моделі на теплому старті (0.046 с проти 0.512 с у хмарної). Це пояснюється відсутністю мережевих затримок (Network RTT): локальна модель працює безпосередньо в оперативній пам'яті пристрою.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Відповідь: Ні, вони протилежні. Локальна модель випереджає хмарну за TTFT на короткому запиті, але суттєво програє за швидкістю генерації (94.3–108.1 проти

1157.6 симв./с на довгому промпті) через обмежену обчислювальну потужність локального CPU/RAM порівняно зі спеціалізованими хмарними прискорювачами.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

Відповідь: На короткому промпті за загальним часом лідирувала локальна модель (0.349 с проти 0.541 с). Проте на довгому промпті хмарна модель згенерувала відповідь майже у 8.9 раза швидше за локальну (2.453 с проти 21.977 с), оскільки при великому обсязі тексту вирішальну роль відіграє швидкість виводу токенів.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: Повтори різняться на 5–15% через фонові процеси ОС, роботу Garbage Collector у Python та мережеві коливання. Одного запуску недостатньо, тому використання медіани кількох запусків є обов'язковим для усунення випадкових сплесків затримки.

5. Чому холодний старт повільніший за теплий?

Відповідь: Під час холодного старту Ollama змушена зчитати вагові коефіцієнти моделі обсягом у кілька гігабайтів з накопичувача (SSD) та ініціалізувати їх у пам'яті. При теплому старті модель уже перебуває в RAM/VRAM і миттєво готова до обчислень.

Питання до чат-бота:

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: Під час TTFT відбуваються передача даних по мережі, токенізація вхідного тексту та Prompt Processing (префіксне обчислення матриць уваги та створення KV-кешу). Окреме вимірювання TTFT важливе для оцінки чутливості інтерфейсу (User Experience), оскільки саме TTFT визначає, як довго користувач чекає на першу реакцію.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: Хмарна модель має додаткову мережеву затримку (RTT) та чергу обробки на сервері, що збільшує TTFT. Проте після старту генерації потужні серверні GPU видають токени зі швидкістю у 10–15 разів вищою за локальний процесор.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: Холодний старт — це первинне завантаження вагових коефіцієнтів моделі з диска в оперативну пам'ять. Повторні запити («теплий старт») виконуються миттєво, бо ваги моделі вже розміщені в ОЗП.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: Медіана стійка до аномальних викликів та викидів (Outliers), викликаних мережевими коливаннями, перепадом навантаження CPU чи фоновими операціями ОС.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: Обробка вхідного промпту обчислюється паралельно для всіх токенів запиту за один прохід ($O(1)$ кроків). Генерація ж відповіді є авторегресійним послідовним процессом: кожен наступний токен вимагає окремого прогону через усі шари нейромережі.

Висновок за завдання 4:

Вимірювання показали фундаментальну відмінність між метриками інференсу: за затримкою до першого токена на теплому старті перемагає локальна модель завдяки відсутності мережевого пересилання (0.046 с). Проте за пропускною здатністю генерації хмарна модель переважає локальну у 11–12 разів на довгих промптах (1157.6 симв./с проти 94.3 симв./с), забезпечуючи підсумковий час виконання 2.453 с проти 21.977 с. Cold start локальної моделі вимагає додаткових ~3.5 с на завантаження ваг у RAM. Для інтерактивного чат-бота найважливішим є TTFT, оскільки він мінімізує час очікування користувача перед порожнім

екраном. Для фонової пакетної обробки великих текстів вирішальною є швидкість генерації (симв./с).

Завдання 5. Оцінювання якості моделей за рубрикою

Проведено комплексне оцінювання відповідей моделей за 6 тестовими завданнями (факти, формат JSON, стиснення, код, логіка, мова) за 3-бальною шкалою рубрики (0, 1, 2 бали).

Таблиця 8 – Оцінка якості за рубрикою

Завдання	**llama3.2:3b**	**qwen3:4b**	**Хмарна модель**
1. Факт	0	0	0
2. Формат	2	2	2
3. Стиснення	0	2	2
4. Код	1	2	1
5. Логіка	2	2	2
6. Українська	0	2	2
Разом /12	5	10	9

Таблиця 9 – Розбіжність оцінок між двома прогонами

Завдання	**llama3.2:3b**	**qwen3:4b**	**Хмарна модель**
1. Факт	ні	ні	ні
2. Формат	ні	ні	ні
3. Стиснення	ні	ні	ні
4. Код	ні	ні	ні
5. Логіка	ні	ні	ні
6. Українська	ні	ні	ні

Аналітичні питання по експерименту:

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: Найбільше балів набрала локальна міркувальна модель qwen3:4b (10/12), випередивши навіть хмарну модель openai/gpt-oss-120b (9/12) та малу інструктивнуllama3.2:3b (5/12). Основна різниця виникла на Завданні 1 (усі моделі згалюцинували рік без RAG), Завданні 3 (стиснення фактів, де llama3.2:3b

втратила більшість чисел), Завданні 4 (дотримання вимоги «лише код» без markdown) та Завданні 6 (якість української мови).

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: Оцінки прогонів розійшлися лише в моделі llama3.2:3b на Завданні 4 (кодування): у першому прогоні модель додала обгортку markdown python, порушивши вимогу «лише код» (1 бал), а у другому видала чистий код (2 бали). Це свідчить про нестабільність дотримання формату в малих інструктивних моделях при повторних викликах.

3. Чи виконила хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: У Завданні 2 всі три моделі видали чистий JSON без блоків markdown. Проте у Завданні 4 вимогу «лише код» повністю без markdown виконала тільки qwen3:4b. Хмарна модель та llama3.2:3b за звичкою обгорнули Python-код у стандартний блок python.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь: Так. Модель llama3.2:3b продемонструвала повну семантичну деградацію у Завданні 6, назвавши стек і чергу «хімічними сполуками та деревними матеріалами з вмістом келлоспіру та фенолу». У Завданні 5 вона використовувала русизми та суржик («ісключення», «Следовательно», «Окончательный ответ»). Моделі qwen3:4b та openai/gpt-oss-120b згенерували бездоганну українську мову.

Детальний аналіз провалу моделі:

• Повний текст запиту: «Чим стек відрізняється від черги? Поясни українською, два абзаци.»

• Повний текст відповіді (без скорочень): "Стек та черга - дві різні горішості, що різниваються за фізичними та хімічними особливостями. Стек зосуджений з горішої частини деревини, що містить більшість хімічних складних сполук. Стек містить такі сполуки як: ерітробіг, келлоспір, келлозолан, фенол."

• Яка правильна відповідь мала бути: Стек — це структура даних типу LIFO («останній зайшов — перший вийшов»), де елементи додаються й видаляються з одного кінця (вершини). Черга — це структура даних типу FIFO («перший зайшов — перший вийшов»), де елементи додаються в кінець, а вилучаються з початку.
• Пояснення, чому так сталося: Під час токенізації слова «стек» малопараметрична модель llama3.2:3b через бідний український датасет зв'язала його не з комп'ютерним терміном (Stack), а з поодинокими текстами про переробку сухий деревних матеріалів чи промислових стіків. Відсутність механізму міркування призвела до генерації повного марення (галлюцинації) із вигаданими виміряними сполуками («ерітробіг», «келлоспір»).
Питання до чат-бота:
1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?
Відповідь: На практиці застосовують стандартизовані бенчмарки (MMLU для ерудиції, HumanEval для коду, GSM8K для математики), підхід «LLM-as-a-Judge» (де більш потужна модель оцінює відповіді молодших моделей), людське A/B тестування (LMSYS Chatbot Arena) та підрахунок статистичних метрик подібності текстів (BLEU, ROUGE, BERTScore).
2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?
Відповідь: Це зумовлено претрейнінгом на масивах коду з GitHub та StackOverflow. У навчальних даних код та структурований JSON майже завжди обгорнуті в блоки json або python. Цей синтаксичний паттерн є стільки стійким у вагах моделі, що він часто перебиває пряму заборону в промпті.
3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?
Відповідь: Головна причина — диспропорція навчального корпусу: англійська мова складає понад 80% інтернету, тоді як українська — частки відсотка. Крім того, токенізатори розбивають кириличні слова на дрібні субодиниці (байти/морфеми), що виснажує контекстне вікно моделі.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: Мовні моделі є ймовірнісними системами. На кожному кроці вибір наступного токена здійснюється шляхом зваженого випадкового семплювання (зважаючи на temperature та top_p), що формує нову траєкторію генерації тексту.

5. Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

Відповідь: Галюцинація — це впевнена генерація неправдивої інформації. Модель робить це тому, що її функція втрат (Loss Function) під час навчання максимізує ймовірність продовження речення, а не перевірку істинності фактів чи виявлення меж власних знань.

Висновок за завданням 5:

Оцінювання за рубрикою виявилося надійним інструментом для виявлення реальних спроможностей моделей. Найпоказовішими для порівняння виявилися завдання на точний факт (Завдання 1, де всі моделі без RAG згалюцинували рік заснування університету) та мовну компетентність. Локальна модель qwen3:4b показала найкращий результат (10/12), продемонструвавши відмінне дотримання формату коду та логічне обчислення. Для роботи з українськомовним текстом та кодом оптимальними є моделі qwen3:4b та openai/gpt-oss-120b, тоді як llama3.2:3b не рекомендована через високий ризик деградації мови та змішування контекстів.

Завдання 6. Керування випадковістю: temperature і top_p

Досліджено вплив параметрів семплювання temperature (0.0, 0.7, 1.5) та top_p (0.1, 0.5, 1.0) на різноманітність відповідей у творчому завданні та точність у фактичному питанні.

Варіант 1:

Придумай назву стартапу в галузі логістики

Рік заснування Національного університету «Одеська Політехніка»"

Таблиця 10 – Вплив параметрів на різноманітність відповідей

Параметр	**Значення**	**Унікальних із 5 (творче)**	**Правильних із 5 (фактичне)**
temperature	0.0	1	0
temperature	0.7	5	—
temperature	1.5	5	0
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	5	—

Таблиця 11 – Перевірка відтворюваності

Показник	**Значення**
Кількість запусків	10
temperature	0.0
Унікальних відповідей	3 (отримано роки: 1992, 1925, 1921)
Усі відповіді однакові (так / ні)	ні

Аналітичні питання по експерименту:

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: Для генерації JSON необхідно встановлювати temperature = 0.0. Це максималізує детермінованість вибору найімовірніших токенів, забезпечує суворе дотримання синтаксису дужок та ключів і зводить до мінімуму випадкові синтаксичні помилки.

2. Яке — генератору варіантів заголовків?

Відповідь: Для генерації заголовків та ідей доцільно використовувати temperature = 0.7 або 1.0. Це сплющує розподіл ймовірностей і дозволяє моделі обирати більш варіативні, креативні та нетривіальні варіанти формулювань.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Відповідь: Параметр temperature математично ділить логіти перед функцією Softmax, масштабуючи й згладжуючи весь розподіл ймовірностей для всіх токенів. Параметр top_p (nucleus sampling) застосовує жорстке динамічне відсікання: він залишає лише найімовірніші токени, чия сумарна ймовірність досягає порогу p, відкидаючи малоймовірний «хвіст».

4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: Ні, temperature = 0.0 не гарантує 100% відтворюваності. В експерименті з 10 запусків модель 7 разів видала «1992», 2 рази «1925» та 1 раз «1921» (отримано 3 унікальні відповіді, прапор однаковості False). Це зумовлено апаратною неасоціативністю плаваючої коми при паралельних обчисленнях на CPU/GPU.

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з'явитися.

Відповідь: Яскравим прикладом із логів експерименту є відповідь: "Національний університет "Майкл Дзенн Полі 技ніку" заснований в 1953 році в Odesy (Tennessee, США)."

Пояснення: При temperature = 1.5 розподіл ймовірностей стає майже рівномірним. Малоймовірні токени (китайські ієрогліфи `技`, американські штати Tennessee, вигадані імена) отримують високий шанс бути обраними на кожному кроці, що призводить до повного розпаду логіки, фактичного марення та мовного змішування.

Питання до чат-бота:

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Відповідь: Він ділить вхідні логіти z_i на значення температури T перед застосуванням Softmax. При різниця між логітами загострюється до максимуму

(найвищий логіт отримує ймовірність приблизно 1), а при високому T розподіл вирівнюється.

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

Відповідь: temperature змінює самі абсолютні ваги ймовірностей усіх кандидатур, тоді як top_p відсікає довгий «хвіст» малоймовірних токенів, впорядковуючи їх за спаданням і залишаючи лише верхню групу з кумулятивною сумою.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: На практиці рекомендується змінювати лише один із них (зазвичай temperature), залишаючи top_p = 1.0. Одночасна зміна ускладнює контроль над генерацією і може викликати непередбачуване погіршення синтаксису.

4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь: Це пояснюється паралелізацією обчислень у багатпотокових середовищах та неасоціативністю операцій із плаваючою комою (Floating-Point Non-Determinism), коли порядок додавання матриць у батчі дає мікроскопічні розбіжності на рівні 15-го знака після коми, що змінює вибір токена-переможця.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: Для витягу структурованих даних (JSON, класифікація, код) використовують діапазон 0.0 – 0.2, а для генерації творчого тексту (копірайтинг, сторітелінг, ідеї) — 0.7 – 1.0

Висновок за завданням 6:

Експеримент підтвердив пряму залежність між параметрами семплювання та детермінованістю тексту. Встановлення temperature = 0.0 є критичним для генерації структурованого JSON та коду. Підвищення температури до 1.5 повністю руйнує мовну зв'язність і призводить до генерації безглуздих слів та змішування алфавітів. Експеримент із 10 запусків довів, що навіть при zero-

temperature система не є 100% детермінованою через специфіку обчислень float, що слід враховувати при тестуванні агентів.

Загальні висновки

Таблиця 12 – Підсумкове порівняння моделей

Критерій	**Локальна мала (3B)**	**Локальна більша (4B)**	**Хмарна**
Швидкість генерації	~71.6 – 108.1 симв./с	~22.0 – 45.0 симв./с	~543.1 – 1157.6 симв./с
TTFT	0.046 – 0.165 с	0.150 – 0.350 с	0.417 – 0.565 с
Якість, балів /12	5	10	9
Стабільність формату (завдання 2)	Низька (порушує вказівку «лише код», вставляє русизми)	Висока (ідеально витримує відсутність markdown та JSON)	Висока (видає чистий JSON, але додає markdown у код)
Потрібні ресурси	~2.0 ГБ VRAM / RAM, слабкий ноутбук	~3.5 ГБ RAM/VRAM, середній ноутбук	Мінімальні локальні ресурси (лише мережа)
Вартість	Безкоштовно (локальний інференс)	Безкоштовно (локальний інференс)	Платне API (або безкоштовні ліміти Groq/OpenAI)
Куди йдуть ваші дані	Залишаються локально на пристрої	Залишаються локально на пристрої	Передаються на зовнішні сервери провайдера
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Для простих рутинних запитів та чат-ботів, де важливі миттєва реакція (TTFT) та автономність	Коли потрібен баланс між складними міркуваннями, якістю та повною конфіденційністю	Коли потрібна висока швидкість генерації великих обсягів тексту (>1000 симв./с), складний аналіз, відсутність галюцинацій та бездоганна мовна грамотність.